

What are the new features introduced in Java 8?

- Default and static methods in Interfaces.
- Lambda Expressions
- @Functional Interface
- forEach() method
- Stream API
- LocalDate , LocalTime, LocalDateTime

MORE VIDEOS



1:32 / 24:07

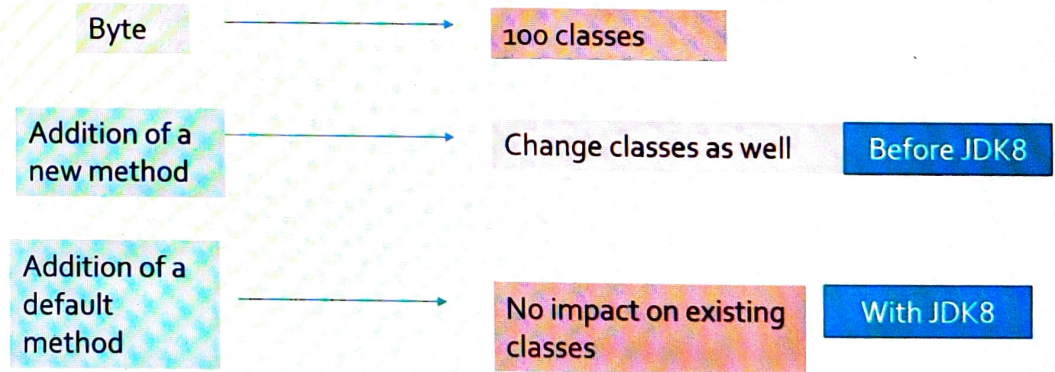


YouTube



Why default method introduced in interfaces?

- Before Java8, interfaces contains only abstract method. All methods are by default public and abstract.
- With Java 8, default method is introduced.
- This helps in achieving addition of a new method in interfaces without need to change existing classes.



Why static method introduced in interfaces?

- If you want to provide a default implementation, which cannot be change .
- Static methods cannot be overridden.
- Access via Interface name or object of class implementing interface.

What is lambda expression?

- Lambda expressions are, basically functions with no name and identifiers.
- Promotes functional programming.
- It helps in expressing instances of single-method class more compactly.

Do you know
what are the
various forms
of writing
lambda
expressions?

- `() -> expression`
- `(parameters) -> expression`
- `(parameters) -> { multiple statements }`

What do you understand by @Functional Interface annotation in java 8?

- @Functional Interface, if used, will force the compiler to check whether the given interface has single-abstract method or not.
- If not, compiler will throw error "**Unexpected @FunctionalInterface annotation**"

How to avoid NullPointerException in java 8?

- Java 8 provides concept of Optionals.
- Optionals can be used to avoid NullPointerException
- Ex : `String value = null;`

What do you mean by stream?

- Streams are just sequence of data from a source.
- With Java8, we can do manipulation on data using stream api
- Example : Youtube is a streaming platform

Can you tell me difference between Collection API and Stream API?

- Collection API:

Elements are not computed on demand basis.

Indexed Access possible.

Iteration was external.

Access of elements was sequential.

- Stream API :

Elements are computed on demand basis.

Indexed access not possible.

Iteration is internal.

Access of elements could be sequential as well as parallel.

Stream does not store data.

An operation on stream does not change its original source.

Streams does not have a fixed size.

There is Employee class having id, name, age field. How to show all employees details.

- **@Data**
- **public class Employee {**
 - private long id;
 - private String name;
 - private long age;
- **}**

```
List<Employee> list = new ArrayList<>();  
// list contains 10000 employees.
```

```
list.stream().forEach( empl -> {  
    System.out.println(empl);  
});
```

Can you show
all employees
from the list,
having age
greater than 30
with java 8?

- **@Data**
- **public class Employee {**
 - private long id;
 - private String name;
 - private long age;
- **}**

*// 3000 employees are greater
than 30 age out of 10000*

```
list.stream().filter( empl -> empl.getAge() > 30 ).forEach( empl -> {  
    System.out.println(empl);  
});
```

Count no. of all employees from the list, having age greater than 30 with java 8?

- **@Data**
- **public class Employee {**
 - private long id;
 - private String name;
 - private long age;
- **}**

// 3000 employees are greater than 30 age out of 10000

```
• list.stream().filter( empl -> empl.getAge() > 30 ).count();
```

Group all employees from the list, with similar names and age greater than 30 in java 8?

- **@Data**
- **public class Employee {**
 - private long id;
 - private String name;
 - private long age;
- **}**

*// 3000 employees are of one same name,
7000 are of other same name out of 10000*

```
list.stream().filter( empl -> empl.getAge() > 30).collect( Collectors.groupingBy (  
empl -> empl.getName() ) )
```

Count no. of all employees from the list, with similar names and age greater than 30 in java 8?

- **@Data**
- **public class Employee {**
 - private long id;
 - private String name;
 - private long age;
- **}**

*// 3000 employees are of one same name,
7000 are of other same name out of 10000*

```
list.stream().filter( empl -> empl.getAge() > 30).collect( Collectors.groupingBy (empl -> empl.getName(), Collectors.counting ))
```